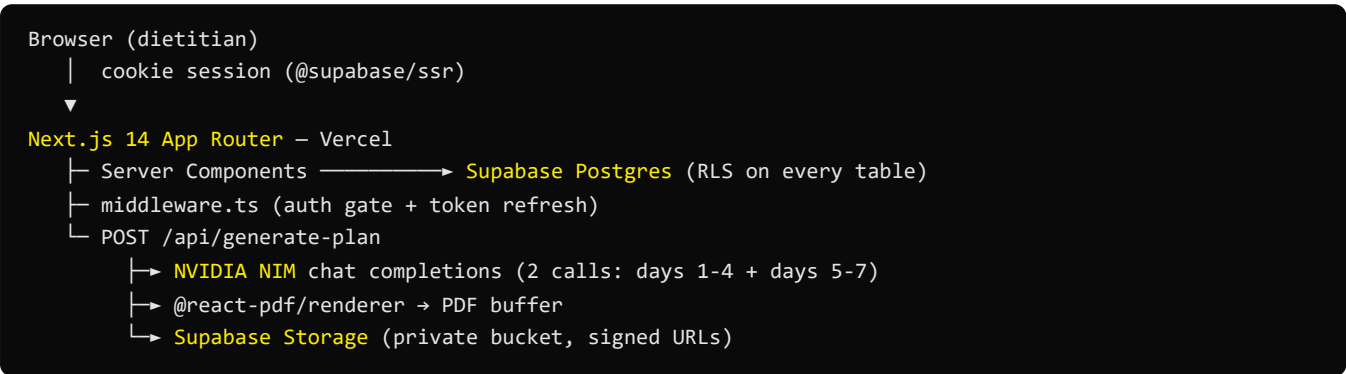


LEANR Diet Platform

Technical Documentation · v1.0 · 10 July 2026



1. ARCHITECTURE OVERVIEW



All database and storage writes execute *as the signed-in user* via the cookie-bound Supabase client — there is no service-role key anywhere, so Row Level Security is the single source of authorization.

Stack

LAYER	TECHNOLOGY
Frontend / SSR	Next.js 14 (App Router), TypeScript, Tailwind CSS (dark LEANR theme, brand yellow #FFED00)
Auth / DB / Files	Supabase — Postgres + Auth + Storage, @supabase/ssr cookie sessions
AI	NVIDIA NIM OpenAI-compatible endpoint — default meta/llama-3.1-70b-instruct , configurable + automatic fallback
PDF	@react-pdf/renderer server-side (renderToBuffer)
Hosting	Vercel (Node runtime; generate route maxDuration = 300)

2. REPOSITORY STRUCTURE

supabase/migrations/0001_init.sql	tables + RLS + signup trigger + storage bucket/policies
scripts/render-sample-pdf.tsx	dev utility: render the PDF layout without calling the AI
src/	
middleware.ts	session refresh + redirect unauthenticated → /login
app/	
login/page.tsx	sign in / sign up
page.tsx	dashboard (client cards, + New Counselling)
counselling/new/page.tsx	first-counselling form (loads draft server-side)
clients/[id]/page.tsx	client detail: latest plan, history, follow-up form
api/generate-plan/route.ts	the generation pipeline (see §5)
auth/signout/route.ts	
lib/	
nim.ts	NIM client, prompts, Zod schemas, fallback/timeout logic
pdf.tsx	premium macro-report PDF document
logo.ts	logo as base64 data URI (embedded in PDFs)
supabase/{client,server}.ts	browser / server Supabase factories
types.ts	IntakeForm, FollowUpInput, row types
components/	CounsellingForm (autosave), FollowUpForm, PlanView, ClientCard, AppHeader, DownloadPdfButton, RegenerateButton

3. DATA MODEL

TABLE	PURPOSE	KEY COLUMNS
profiles	One row per dietitian; auto-created by trigger on_auth_user_created	id (= auth.users.id), full_name
clients	Clients owned by a dietitian; full intake retained	dietitian_id, basics columns, intake JSONB
diet_plans	One row per generated weekly plan	client_id, week_number, source (first_counselling follow_up), plan JSONB, pdf_path
followups	Weekly check-in used to generate the next plan	client_id, week_number, weight_kg, adherence, data JSONB
form_drafts	Live autosave of the counselling form	unique (dietitian_id, kind), data JSONB

RLS: every table has `FOR ALL USING (dietitian_id = auth.uid()) WITH CHECK (dietitian_id = auth.uid())` (profiles: `id = auth.uid()`). **Storage:** private bucket `diet-pdfs`; object paths are `<dietitian_id>/<client_id>/week-N-<ts>.pdf` and policies require the first folder segment to equal `auth.uid()`. Downloads use 1-hour signed URLs created client-side (RLS applies).

4. AUTHENTICATION FLOW

- `src/middleware.ts` runs on every request: refreshes the Supabase session cookie and redirects unauthenticated users to `/login` (and authenticated users away from it).
- Server components and route handlers build a per-request client via `lib/supabase/server.ts` (cookie-bound). The browser uses `lib/supabase/client.ts`.
- Signup passes `full_name` in user metadata; a `SECURITY DEFINER` trigger copies it into `profiles`.

5. AI GENERATION PIPELINE (SRC/LIB/NIM.TS)

The shared NIM serving functions cap completions at roughly 2k tokens — a full 7-day plan with per-meal macros does not fit in one response. The plan is therefore generated in **two calls** and merged:

1. **Part 1** — overview (summary, daily calories, macro targets, guidelines, hydration, foods to avoid) + days 1–4.
2. **Part 2** — days 5–7; the prompt includes a compact listing of days 1–4 meals so menus vary, plus the part-1 calorie/macro targets for consistency.

Each call goes through the same hardened path:

- **Constrained decoding:** `response_format: {type: "json_object"}`; degrades gracefully (one retry without it) if a model rejects the parameter, same for `max_tokens` above a model's cap.
- **Timeout:** `AbortSignal.timeout(120s)` per call — never waits on NVIDIA's multi-minute gateway 504s.
- **Fallback model:** any transient failure (timeout, 429, 5xx, network) retries once on `NVIDIA_FALLBACK_MODEL` (default `meta/llama-3.1-8b-instruct`).
- **Validation:** responses are parsed (code-fence tolerant) and validated with Zod (`PartOneSchema` / `PartTwoSchema`; merged result re-validated against `DietPlanSchema` — exactly 7 days). One corrective retry feeds validation errors back; truncated responses instead get a fresh brevity instruction (the broken output is *not* echoed back).
- **Hard rules in the system prompt:** diet-type enforcement (vegan/veg/eggetarian/non-veg), absolute exclusion of allergies/intolerances/dislikes, adaptation to medical conditions, per-meal macros that sum to daily totals ($\pm 5\%$).

Plan JSON contract (stored in `diet_plans.plan`)

```
{ "summary": string, "daily_calories": number,
  "macros": { "protein_g": n, "carbs_g": n, "fat_g": n },
  "guidelines": string[], "hydration": string, "foods_to_avoid": string[],
  "days": [ 7 × { "day": "Day N", "total_calories": n, "meals": [
    { "name": "Breakfast", "time": "08:00",
      "items": [{ "food": string, "quantity": string }], "notes": string,
      "calories": n, "protein_g": n, "carbs_g": n, "fat_g": n } ] } ] }
```

6. API REFERENCE — POST /API/GENERATE-PLAN

Auth: session cookie (401 otherwise). Runtime: Node, `maxDuration 300`. Three request variants:

BODY	BEHAVIOUR
<code>{"type": "first", "form": IntakeForm}</code>	Creates the client, generates Week 1, renders + stores PDF, inserts plan row, clears the autosaved draft. Returns <code>{clientId, planId, week}</code> .
<code>{"type": "followup", "clientId", "followup": FollowUpInput}</code>	Inserts the follow-up row, updates client weight, generates week <code>latest+1</code> seeing the previous menu.
<code>{"type": "regenerate", "clientId"}</code>	Retries generation from stored intake (used when a first generation failed after the client was saved).

Errors: 400 invalid body · 401 unauthenticated · 404 unknown client · 502 with `{error, clientId?}` when generation/PDF/storage fails — the UI uses `clientId` to route to the client page where retry is offered. The PDF filename pattern is `week-N-<timestamp>.pdf` ; plan start date is the day after generation (day labels on the PDF are real dates).

7. PDF RENDERING (`SRC/LIB/PDF.TSX`)

- Premium macro-report layout: black header band with client name + LEANR logo (embedded data URI from `lib/logo.ts`), macro target cards, color legend, per-day tables (Time | Meal | Foods | Calories | Protein | Carbs | Fat | Cal%), highlighted Daily Total rows, Weekly Summary with P%/C%/F%, guidelines, foods to avoid, and a two-line disclaimer footer on every page.
- Daily totals, Cal% and the weekly summary are *computed from meal data*, never trusted from the model.
- `next.config.mjs` marks `@react-pdf/renderer` as a server external package (required for the Node runtime bundle).
- Preview the layout without the AI: `npx tsx scripts/render-sample-pdf.tsx out.pdf`.

8. AUTOSAVE DESIGN

`CounsellingForm` debounces every state change (1.2s) into an upsert on `form_drafts` keyed by `(dietitian_id, kind)` , with a live Saving/Saved/Failed indicator. The draft is fetched server-side on page load (no flash) and deleted only after a successful generation, so a failed generation or closed tab never loses call notes.

9. ENVIRONMENT VARIABLES

VARIABLE	SCOPE	NOTES
<code>NEXT_PUBLIC_SUPABASE_URL</code>	client + server	Supabase project URL
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	client + server	Safe to expose — RLS enforces access
<code>NVIDIA_API_KEY</code>	server only	From build.nvidia.com; never sent to the browser
<code>NVIDIA_MODEL</code>	server, optional	Default <code>meta/llama-3.1-70b-instruct</code> ; <code>mistralai/mistral-small-4-119b-2603</code> is a fast, strong alternative when the default is congested
<code>NVIDIA_FALLBACK_MODEL</code>	server, optional	Default <code>meta/llama-3.1-8b-instruct</code> ; used automatically on transient failures
<code>NVIDIA_NIM_URL</code>	server, optional	Default <code>https://integrate.api.nvidia.com/v1/chat/completions</code>

10. SETUP & DEPLOYMENT

1. **Supabase:** create a project; run `supabase/migrations/0001_init.sql` in the SQL Editor (creates all tables, RLS, trigger, bucket, policies). Recommended: disable "Confirm email" under Authentication → Sign In / Providers → Email (the built-in mailer is rate-limited to ~2 emails/hour).
2. **Local:** `npm install` → copy `.env.example` to `.env.local` and fill the three keys → `npm run dev`.
3. **Vercel:** import the GitHub repo, set the env vars, deploy; then add the Vercel domain to Supabase Authentication → URL Configuration.

11. OPERATIONS & TROUBLESHOOTING

SYMPTOM	CAUSE / FIX
Generation slow or 502 with "NVIDIA NIM ... failed (504)"	Shared free NIM endpoint congested. The fallback model usually still succeeds; optionally set <code>NVIDIA_MODEL</code> to a healthier model. Retry from the client page — intake is never lost.
"AI returned an invalid diet plan after 2 attempts"	Rare double validation failure — retry. Persistent cases: check the model supports <code>response_format</code> and ~2k-token outputs per half-plan.
"Email rate limit exceeded" on signup	Supabase's built-in mailer (~2/hour). Disable Confirm email, or delete the stuck unconfirmed user and retry.
PDF download does nothing	Pop-up blocked, or storage policies missing — re-run the migration section for <code>storage.objects</code> policies.
401 from NIM	Bad/missing <code>NVIDIA_API_KEY</code> .

Security summary: no service-role key in the codebase; all reads/writes ride the user's JWT through RLS; the private bucket is only reachable through per-user policies and short-lived signed URLs; the only server-held secret is the NVIDIA key.